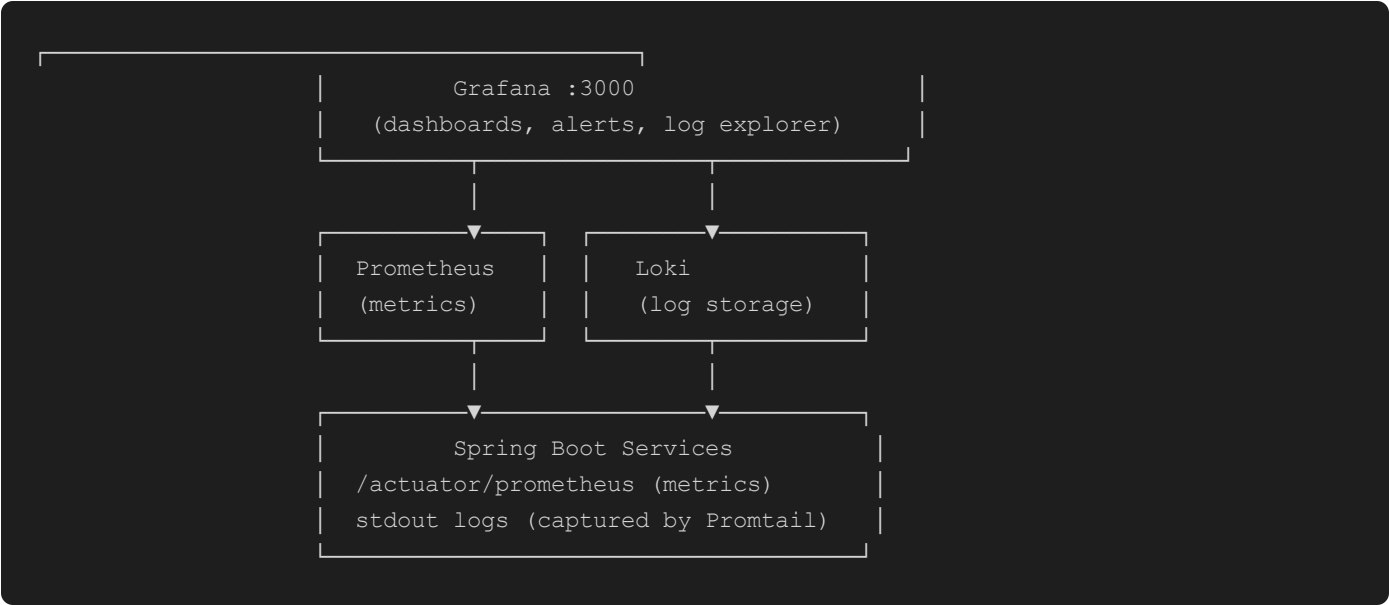


Monitoring & Logs

Stack Overview



Tool	Port	Role	AWS Equivalent
Prometheus	9090	Scrapes metrics every 15s	CloudWatch Metrics
Grafana	3000	Dashboards + alerts	CloudWatch Dashboards
Loki	3100	Stores log lines	CloudWatch Logs
Promtail	DaemonSet	Ships logs to Loki	CloudWatch Agent

Accessing Grafana

Port-forward from K8s
kubectl port-forward svc/grafana 3000:3000 -n monitoring

Open in browser

open http://localhost:3000

```
Default login: admin / admin
```

Metrics (Prometheus)

Every Spring Boot service exposes metrics at `/actuator/prometheus`.

Prometheus scrapes them every 15s. Example metrics:

```
# HTTP request rate
http_server_requests_seconds_count{uri="/api/orders",status="200"}
```

JVM memory

```
jvm_memory_used_bytes{area="heap"}
```

Kafka consumer lag

```
kafka_consumer_records_lag{topic="order.created",partition="0"}
```

HikariCP (DB connection pool)

```
hikaricp_connections_active{pool="HikariPool-order-service"}
```

Key Grafana dashboards to set up:

- JVM Micrometer (Dashboard ID: 4701 — import from grafana.com)
- Spring Boot Statistics (Dashboard ID: 12900)
- Kafka Consumer Lag (Dashboard ID: 12554)

Logs (Loki + Promtail)

How logs flow:

```
Pod stdout → Promtail DaemonSet → Loki → Grafana (query with LogQL)
```

Promtail runs as a **DaemonSet** (one pod per Kubernetes node).

It watches `/var/log/pods/ecommerce_*/` and ships every log line to Loki.

Querying Logs in Grafana

Go to **Explore** → select **Loki** data source.

```
# All logs from order-service  
{namespace="ecommerce", app="order-service"}
```

Only ERROR lines

```
{namespace="ecommerce"} |= "ERROR"
```

Kafka-related logs

```
{namespace="ecommerce", app="order-service"} |= "order.created"
```

Failed payment events

```
{namespace="ecommerce", app="payment-service"} |= "payment.failed"
```

Last 100 lines from user-service

```
{app="user-service", namespace="ecommerce"} | limit 100
```

Correlation IDs (Tracing Requests)

Kong adds a `X-Request-ID` header to every request.

Spring Boot services log this ID. You can trace a single request across all services:

```
# Make a request
curl -H "Content-Type: application/json" \
  -X POST http://localhost:8000/api/orders ... \
  -v 2>&1 | grep "X-Request-ID"
```

→ **X-Request-ID: a1b2c3d4-e5f6**

Then search Loki for all logs with that ID

```
{namespace="ecommerce"} |= "a1b2c3d4-e5f6"
```

Shows the full journey: kong → order → kafka → inventory → payment → notification

Alerting

In Grafana, create alert rules under **Alerting → Alert Rules**:

Alert: High Error Rate

Condition: `rate(http_server_requests_seconds_count{status=~"5.."}[5m]) > 0.1`

For: 2 minutes

→ Send to: Slack channel / email

Alert: Pod Down

Condition: `kube_deployment_status_replicas_available{namespace="ecommerce"} < 1`

For: 1 minute

→ Send to: PagerDuty / Slack

Alert: Kafka Consumer Lag

Condition: `kafka_consumer_records_lag > 1000`

For: 5 minutes

→ Payment or inventory consumers are falling behind

vs CloudWatch (Real AWS)

This stack is our replacement for CloudWatch. Feature mapping:

Floci Stack	Real AWS
Promtail ships logs to Loki	CloudWatch Agent ships logs to CloudWatch Logs
<code>{app="order-service"}</code> in LogQL	<code>/ecommerce/order-service</code> log group in CloudWatch
Prometheus metrics in Grafana	CloudWatch Metrics + dashboards
Grafana alerts	CloudWatch Alarms → SNS → email/Lambda
Loki retention: 30 days (configurable)	CloudWatch Logs retention: 1 day to 10 years

Key difference: On real AWS, you'd add this to your `application.yml` :

```
management:
  cloudwatch:
    metrics:
      export:
        namespace: Ecommerce/OrderService
        enabled: true
```

Our setup achieves the same observability locally with no AWS costs.

Checking Logs Without Grafana (Quick Debug)

```
# Stream logs live
kubectl logs -n ecommerce -l app=order-service -f
```

Last 100 lines

```
kubectl logs -n ecommerce -l app=order-service --tail=100
```

Logs since a time

```
kubect1 logs -n ecommerce -l app=order-service --since=10m
```

All services at once (requires kubetail or stern)

```
kubectrl logs -n ecommerce -l tier=app -f --prefix
```

docker-compose equivalent

```
docker compose logs -f order-service --tail=50
```

Health Checks

All Spring Boot services expose:

```
# Is the service up?  
curl http://localhost:8083/actuator/health
```

```
→ { "status": "UP", "components": { "db": { "status": "UP" }, "kafka": { "status": "UP" } } }
```

Is it ready for traffic?

```
curl http://localhost:8083/actuator/health/readiness
```

Detailed metrics endpoint (scraped by Prometheus)

```
curl http://localhost:8083/actuator/prometheus | head -30
```

In Kubernetes, the readiness probe on each pod uses `/actuator/health/readiness`.

Kong only routes to a pod after its readiness probe passes. This ensures zero-downtime deployments — new pods only receive traffic when fully started.